Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования «Уральский федеральный университет
имени первого Президента России Б. Н. Ельцина»
Институт радиоэлектроники и информационных технологий — РТФ
Центр ускоренного обучения

ТЕХНОЛОГИИ ПРОГРАММИРОВАНИЯ

# Лабораторная работа 6

REST сервис и база данных H2

Выполнил Устьянцев М. Д.

Группа РИЗ-250213у

Номер в списке группы 18

Екатеринбург 2026

## 1 Цель работы

Разработать REST-сервис на Spring Boot, подключить файловую базу H2 и реализовать операции создания, чтения, изменения и удаления данных. Проверить работу сервиса автоматическими тестами и ручными запросами в Postman.

## 2 Описание задачи

Реализованы таблица STUDENTS из методички и дополнительная таблица DISCIPLINES. Для каждой сущности созданы контроллер, DAO и сервисный слой. Все методы возвращают признак успеха, сообщение и данные; ошибочные запросы получают соответствующий HTTP-статус.

Проект labwork6 использует Java 17, Spring Boot 3.5.16, Spring Web, Spring Data JPA, Lombok, Jakarta Validation и H2 2.3.232. Версия Spring Boot согласована с предыдущей работой. Пакеты controller, service, dao и entity разделяют HTTP-обработку, бизнес-логику, доступ к данным и модель.

## 3 Контракт сервиса

| Метод | Маршрут | Результат |
|---|---|---|
| GET | /api/students | Список студентов |
| GET | /api/students/{id} | Студент по id |
| POST | /api/students | Создание, HTTP 201 |
| PUT | /api/students | Изменение, id в JSON |
| DELETE | /api/students/{id} | Удаление, ответ с id |

Для учебных дисциплин используются те же методы с префиксом /api/disciplines. Для второй таблицы выбраны поля name, semester и hours: название, семестр и количество учебных часов. PUT изменяет существующую запись и не создаёт новую при неизвестном id.

## 4 Подключение и схема базы данных

База хранится в testdb/students.mv.db внутри проекта. Подключение настроено в application.properties; схему создаёт schema.sql. Hibernate проверяет соответствие сущностей таблицам, а open-in-view отключён. Данные сохраняются после остановки приложения.

```
spring.datasource.url=jdbc:h2:file:./testdb/students;DB_CLOSE_ON_EXIT=FALSE
spring.jpa.open-in-view=false
spring.jpa.hibernate.ddl-auto=validate
spring.sql.init.mode=always
```

Код создания таблицы студентов:

```
CREATE TABLE IF NOT EXISTS STUDENTS (
   id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   name VARCHAR(15) NOT NULL CHECK (CHAR_LENGTH(TRIM(name)) > 0),
   surname VARCHAR(25) NOT NULL CHECK (CHAR_LENGTH(TRIM(surname)) > 0),
   faculty VARCHAR(20) NOT NULL CHECK (CHAR_LENGTH(TRIM(faculty)) > 0),
   age INT NOT NULL CHECK (age BETWEEN 1 AND 120)
);
```

Код дополнительной таблицы учебных дисциплин:

```
CREATE TABLE IF NOT EXISTS DISCIPLINES (
   id INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   name VARCHAR(100) NOT NULL CHECK (CHAR_LENGTH(TRIM(name)) > 0),
   semester INT NOT NULL CHECK (semester BETWEEN 1 AND 12),
   hours INT NOT NULL CHECK (hours BETWEEN 1 AND 2000)
);
```

Профиль demo добавляет два примера студентов и две дисциплины SQL-скриптом demo-data.sql. Повторный обычный запуск не восстанавливает удалённые записи. Для DBeaver используется абсолютный путь к той же базе, пользователь sa и пустой пароль; перед подключением приложение останавливается.

## 5 Реализация доступа к данным

Сущности Student и Discipline снабжены аннотациями @Entity и @Table. Первичный ключ генерируется H2 через GenerationType.IDENTITY. Имена и длины колонок соответствуют SQL-схеме. Lombok создаёт конструкторы, методы чтения и изменения полей.

DAO реализован через EntityManager. Ниже приведены методы StudentDAOImpl; DisciplineDAOImpl построен аналогично.

```
public List<Student> getAllStudents() {
   return entityManager.createQuery("from Student order by id", Student.class).getResultList();
}

@Override
public Student getStudent(int id) {
   return entityManager.find(Student.class, id);
}

@Override
public Student saveStudent(Student student) {
   if (student.getId() == null) {
      entityManager.persist(student);
      return student;
   }
   return entityManager.merge(student);
}

@Override
public void deleteStudent(Student student) {
   entityManager.remove(student);
}
```

persist сохраняет новую сущность, merge обновляет существующую, find читает запись по первичному ключу, remove удаляет управляемую сущность. Получение списка выполняется JPQL-запросом с сортировкой по id.

## 6 Сервисный слой и контроллеры

Сервис проверяет наличие записи и задаёт границы транзакций. Чтение помечено readOnly=true, операции записи выполняются с @Transactional. При исключении изменения откатываются. Код обновления студента:

```
public Student updateStudent(Student student) {
   if (student.getId() == null) throw new IllegalArgumentException("PUT requires id");
   // Check existence before updating so PUT cannot create a missing resource.
   Student existing = getStudent(student.getId());
   existing.setName(student.getName());
   existing.setSurname(student.getSurname());
   existing.setFaculty(student.getFaculty());
   existing.setAge(student.getAge());
   return dao.saveStudent(existing);
}
```

POST возвращает HTTP 201, заголовок Location и созданную запись. В контроллере используются разные группы валидации для создания и изменения:

```
public ResponseEntity<ApiResponse<Student>> create(
      @RequestBody @Validated({ValidationGroups.Create.class, Default.class}) StudentRequest
request) {
   Student saved = service.createStudent(request.toEntity());
   return ResponseEntity.created(URI.create("/api/students/" + saved.getId()))
         .body(ApiResponse.ok("Student created", saved));
}
```

DELETE возвращает HTTP 200 и идентификатор удалённой записи. ResourceNotFoundException преобразуется в HTTP 404. ApiExceptionHandler формирует единый ответ при ошибках, а непредвиденные исключения записывает в журнал без передачи технических деталей клиенту.

## 7 Проверка работы

14 октября 2026 года сборка Maven verify завершилась успешно. Выполнено 64 интеграционных теста, сбоев, ошибок и пропусков нет. Проверяются CRUD обеих таблиц, отсутствующие записи, типы JSON, границы значений, независимость таблиц и откат транзакции при нарушении ограничения базы данных.

| **Проверка** | **Результат** |
|---|---|
| Maven verify | 64 теста, все успешны |
| Реальные HTTP-запросы | 41 запрос, 205 проверок |
| Файловая база H2 | Два перезапуска JVM, изменения сохранены |
| Postman | Ручные CRUD, HTTP 400 и 404 |

Скрипт scripts/smoke_test.py запускает собранный JAR, выполняет HTTP-запросы и дважды перезапускает процесс. Первый перезапуск подтверждает сохранение обновлённых данных, второй — сохранение удалений. Используется отдельная тестовая база в target; рабочая база не изменяется. Доказательства сохранены в target/http-results.json.

Успешный ответ содержит success=true. Пример ответа при попытке получить удалённую дисциплину:


```
{
   "success": false,
   "message": "Discipline with id=4 was not found",
   "data": null
}
```


Коллекция Postman содержит 26 запросов с проверками статусов и данных. Автоматические HTTP-проверки выполнены Python-скриптом; ручные запросы отправлены через Postman Lightweight API Client. Скриншоты фактических ответов приведены далее.

# 8 Ручное тестирование в Postman

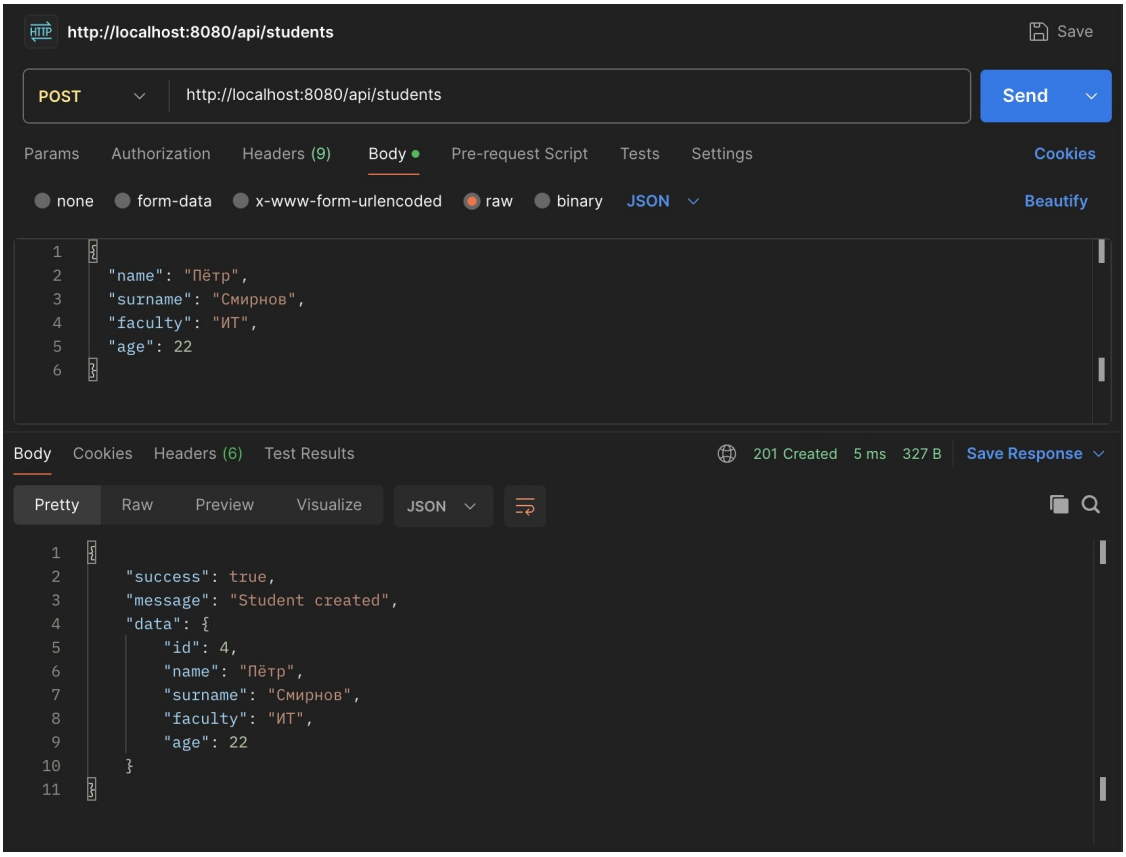



Рисунок 1 — Создание студента POST и ответ 201 Created

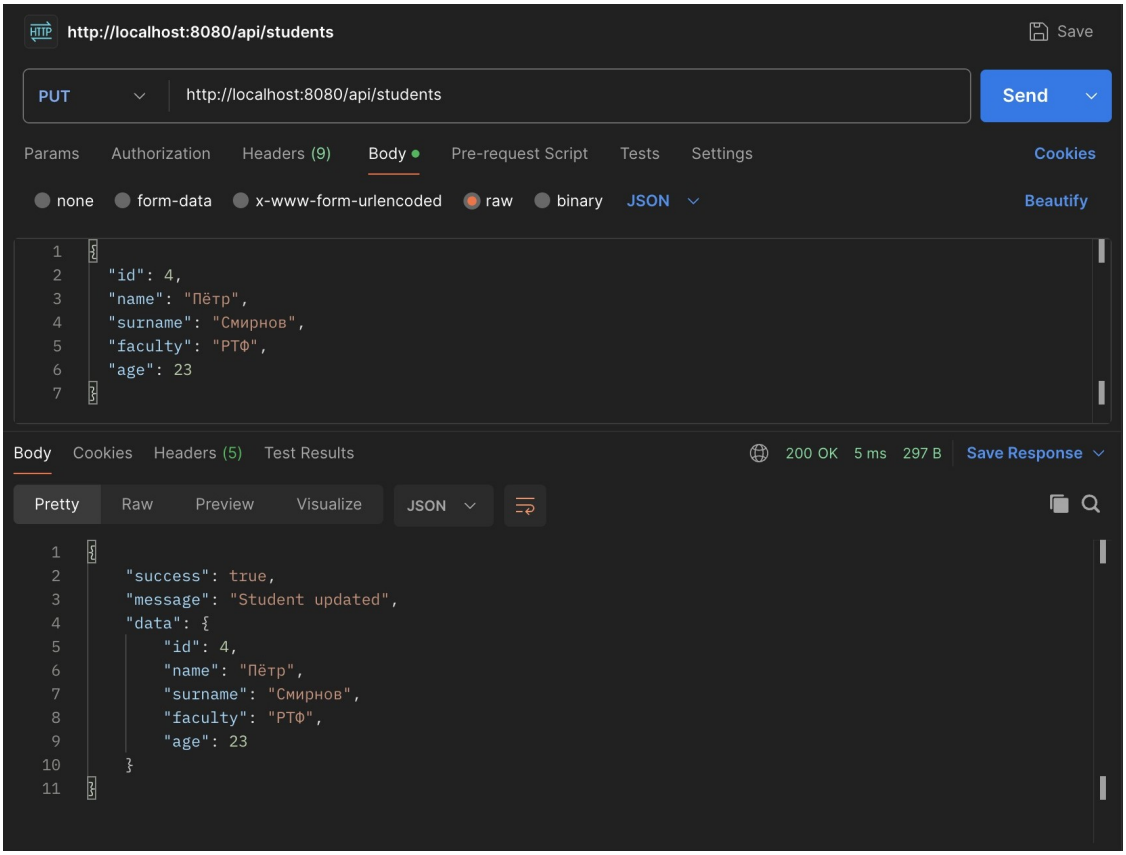



Рисунок 2 — Изменение студента PUT и ответ 200 OK

## Ручное тестирование в Postman

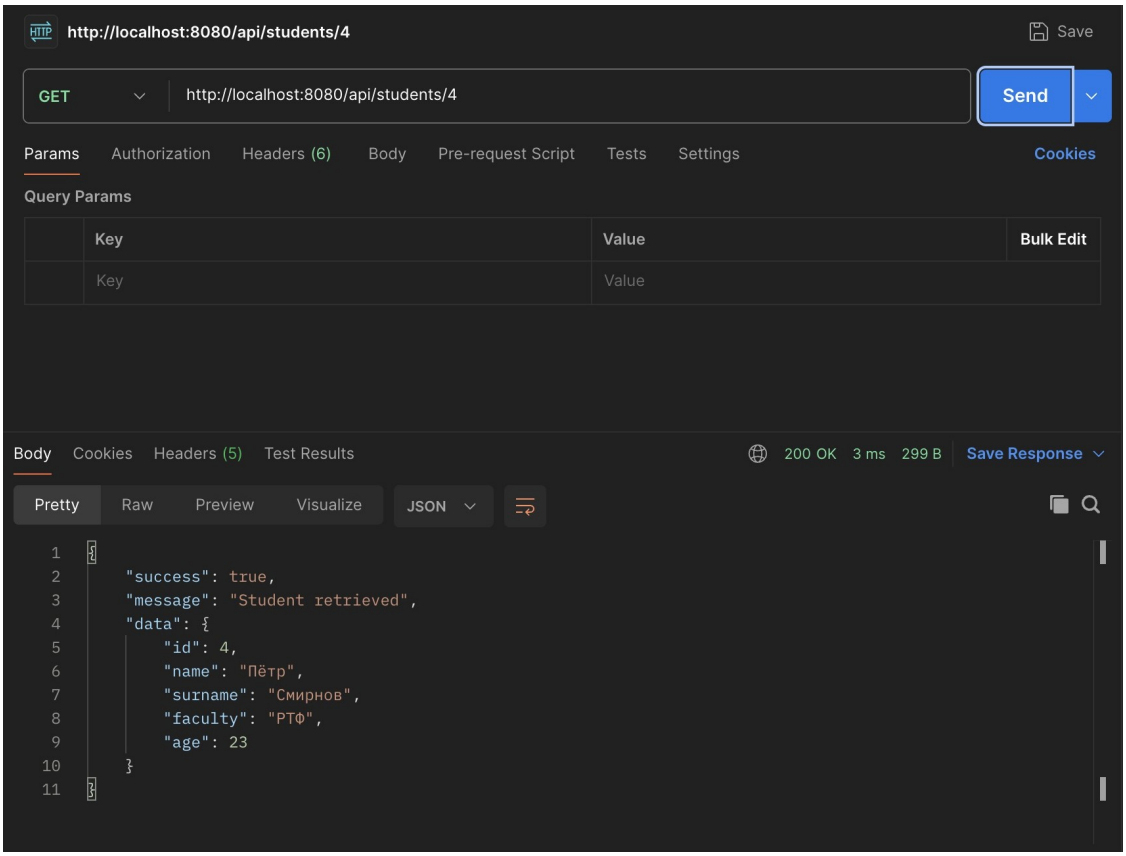



Рисунок 3 — Чтение изменённого студента GET

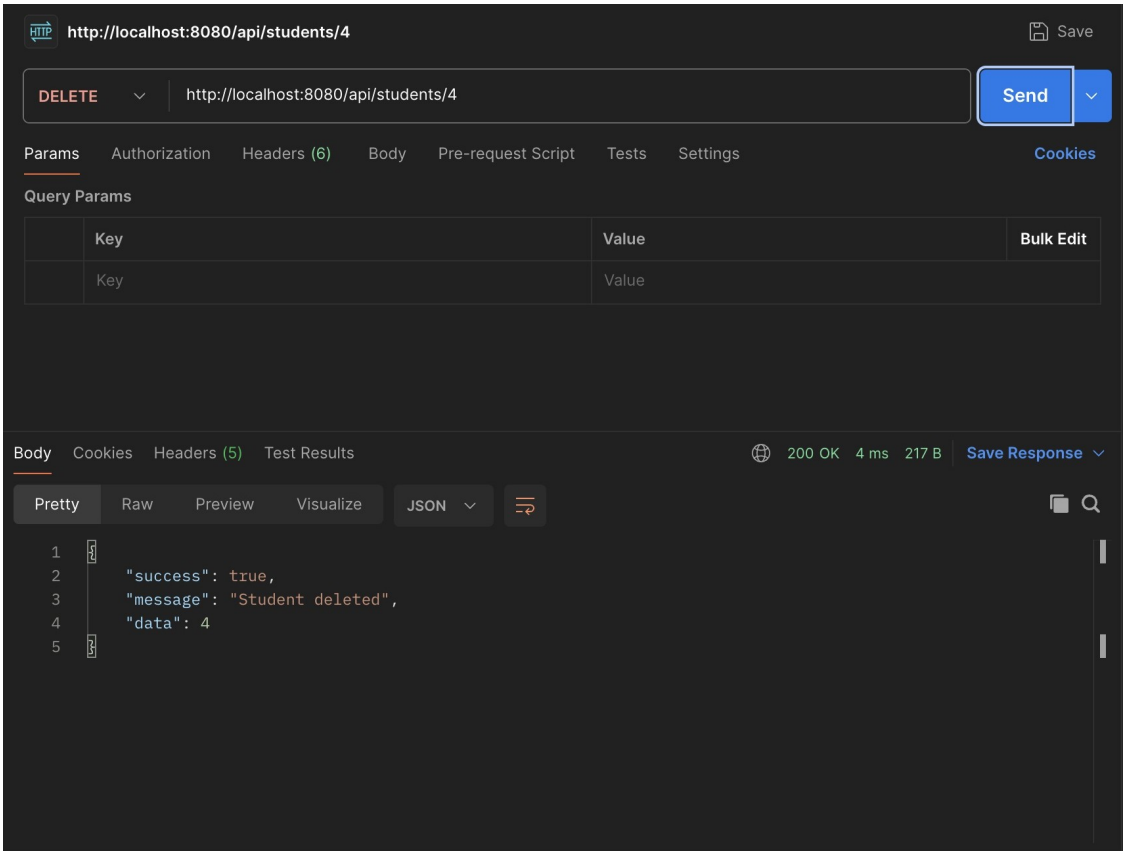



Рисунок 4 — Удаление студента DELETE и подтверждение операции

## Ручное тестирование в Postman

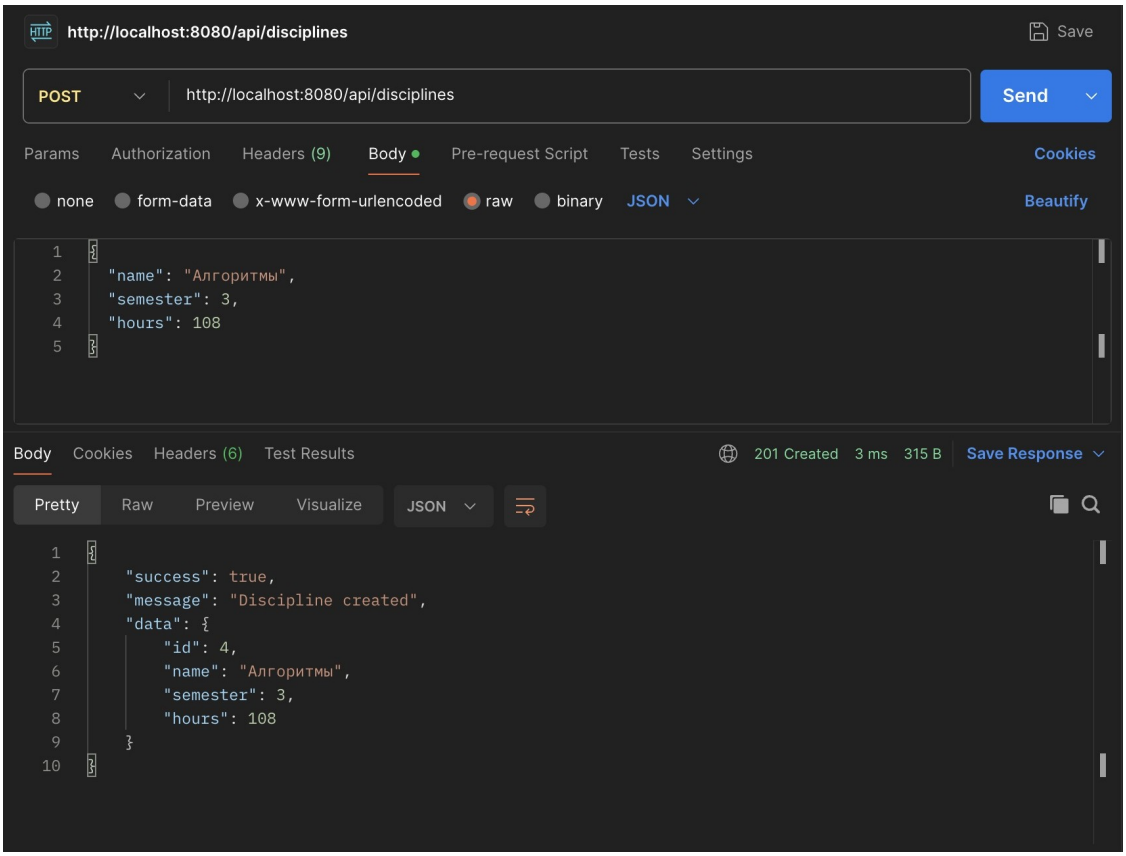



Рисунок 5 — Создание учебной дисциплины POST

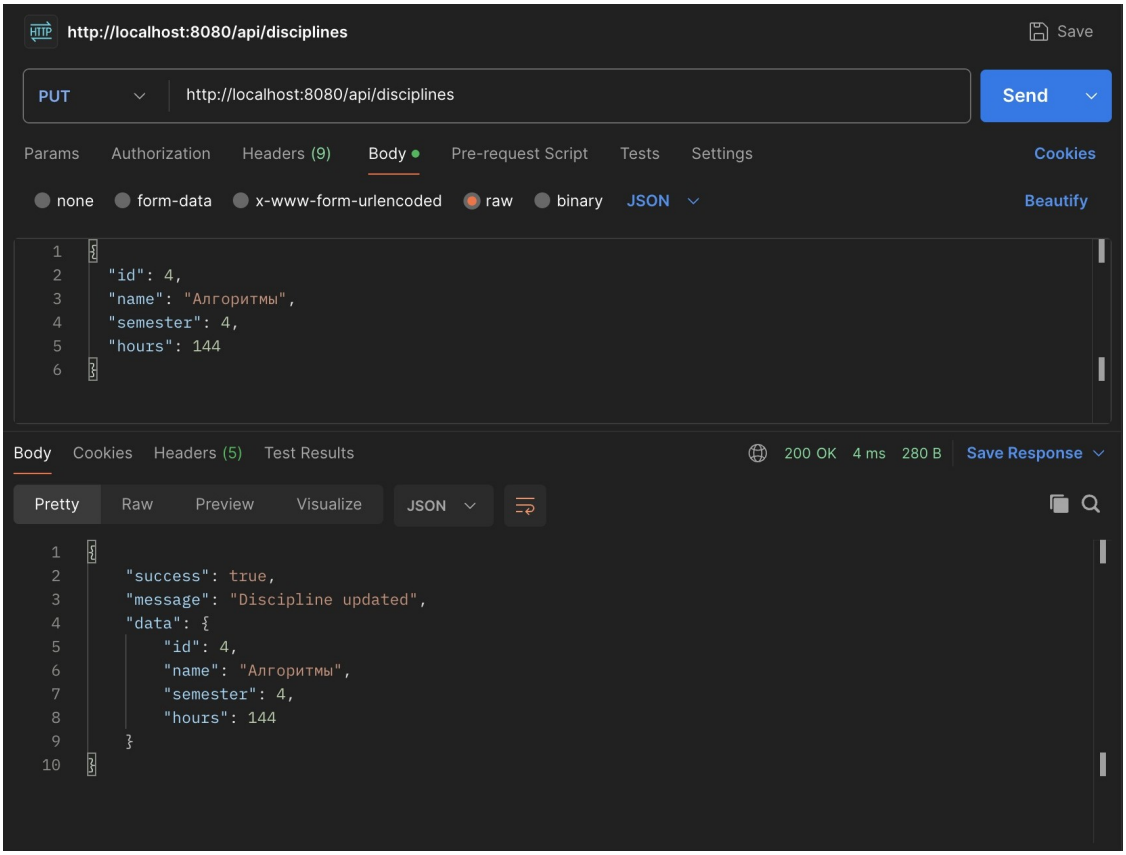



Рисунок 6 — Изменение семестра и часов дисциплины PUT

## Ручное тестирование в Postman

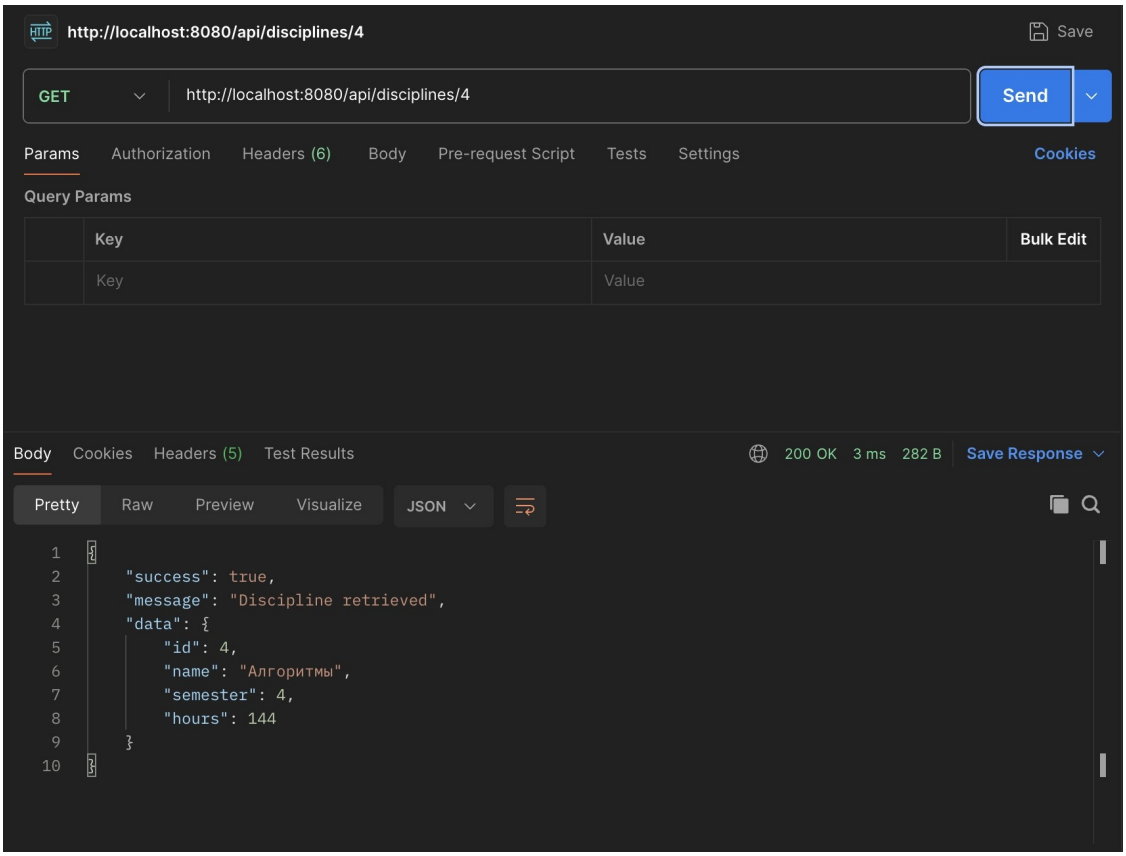



Рисунок 7 — Чтение изменённой дисциплины GET

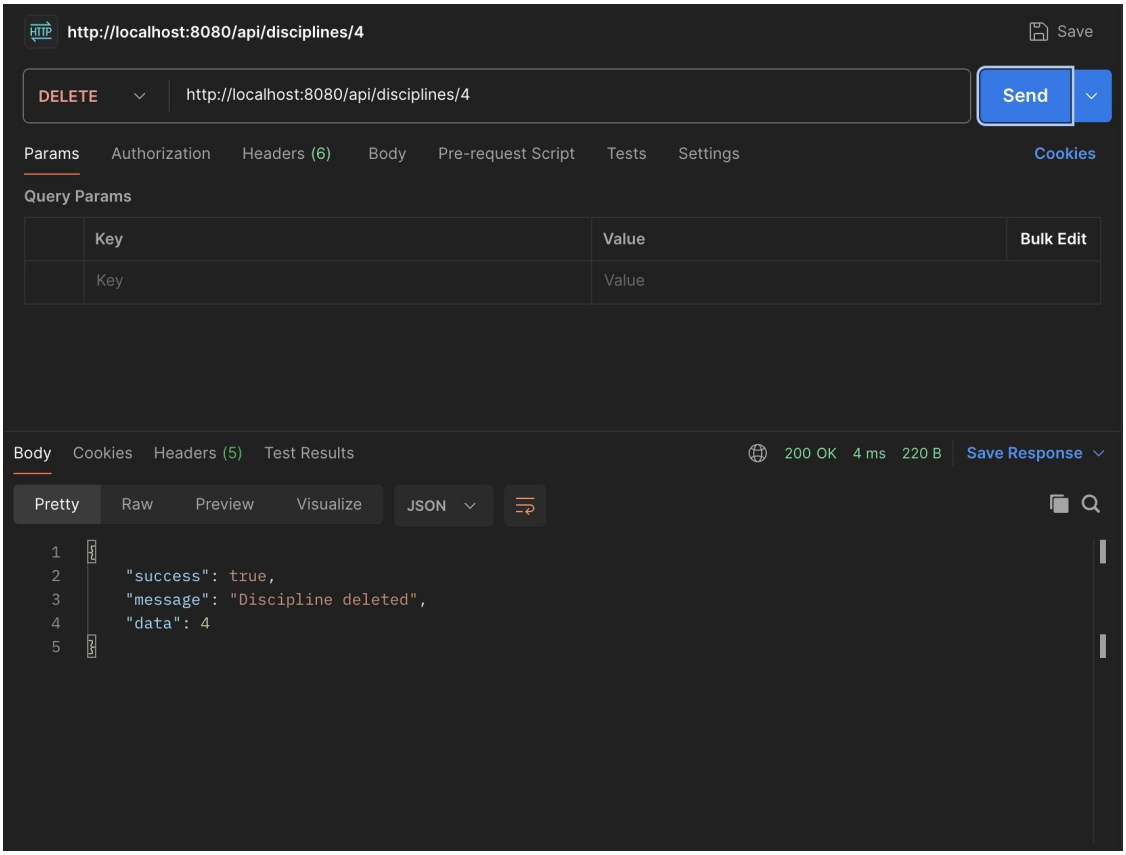



Рисунок 8 — Удаление учебной дисциплины DELETE

## Ручное тестирование в Postman

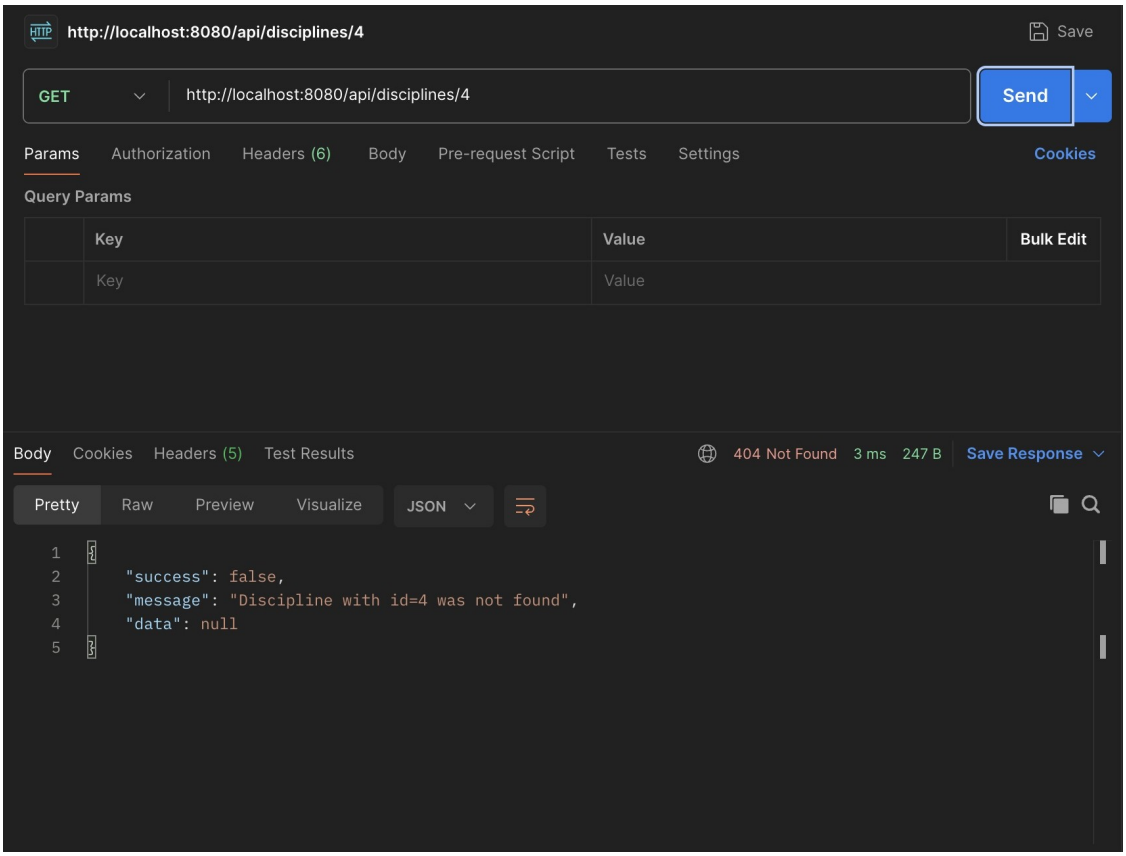



Рисунок 9 — Чтение удалённой дисциплины и ответ 404

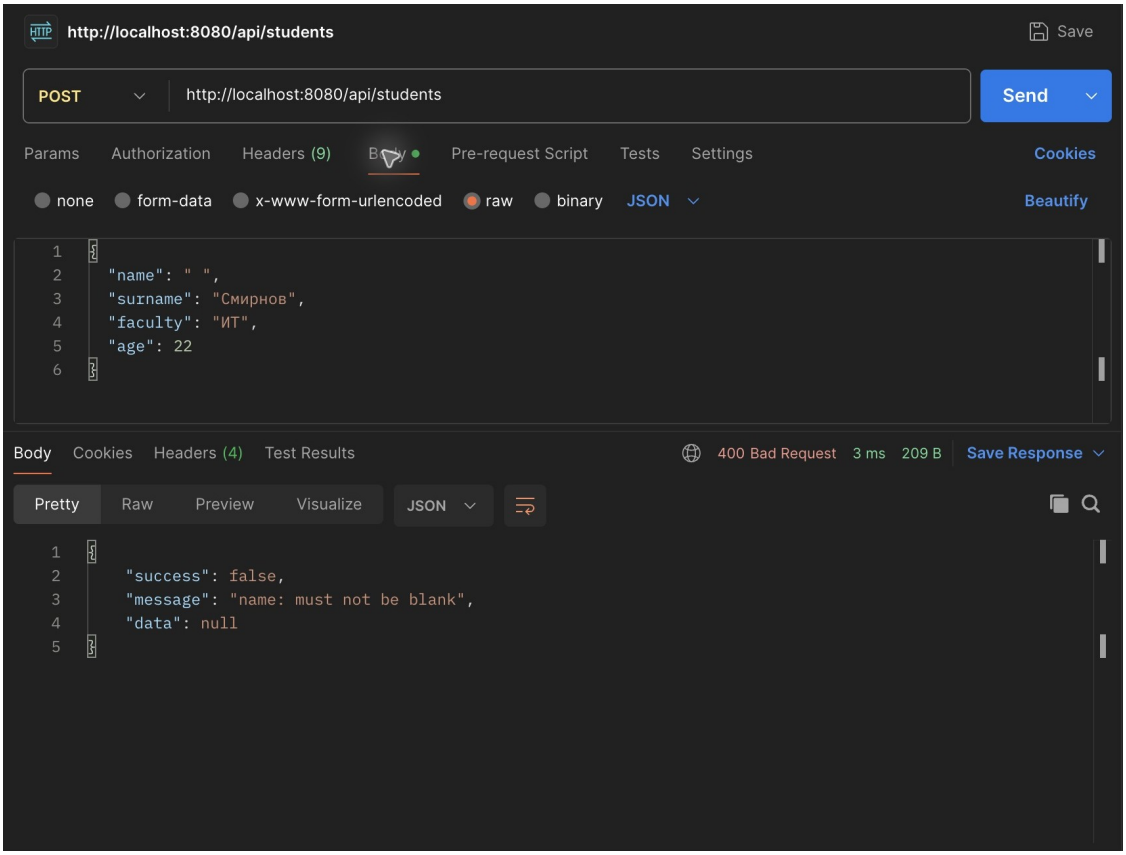



Рисунок 10 — Пустое имя студента и ответ 400

## 9 Ответы на чётные вопросы

Номер в списке группы 18, поэтому даны ответы на вопросы 2, 4 и 6.

## Вопрос 2 Подходы API First и Code First

API-First предполагает, что интерфейс взаимодействия проектируют в начале разработки: согласуют ресурсы, операции, структуры сообщений, статусы и ошибки. Описание API, например в формате OpenAPI, служит контрактом для сервера, клиентов и тестов. Это позволяет разрабатывать компоненты параллельно и заранее проверить согласованность интерфейса.

При Code-First сначала пишут реализацию, а описание API получают из кода и аннотаций. Такой подход удобен для небольшого проекта, но требует контроля, чтобы изменения реализации не нарушали ожидания клиентов. API-First шире простого написания файла спецификации: он задаёт приоритет интерфейса в процессе разработки. В данной работе маршруты заданы методичкой, а реализация выполнена в контроллерах.

## Вопрос 4 Виды интеграции информационных систем

Одна из распространённых классификаций выделяет четыре способа. Передача файлов — системы обмениваются пакетами данных, например CSV или XML, по расписанию. Подход прост, но изменения поступают с задержкой. Общая база данных — приложения читают и записывают общую схему; это обеспечивает доступ к одним данным, но связывает системы со структурой базы.

Удалённый вызов — приложение обращается к API другого приложения, например через REST, SOAP или gRPC, и обычно ожидает ответ. Обмен сообщениями — системы передают команды и события через брокер; обработка может быть асинхронной, а отправитель и получатель меньше зависят от одновременной доступности друг друга. На практике эти способы сочетают; для сообщений требуется учитывать повторную доставку и идемпотентность обработки.

## Вопрос 6 gRPC и отличие от REST

gRPC — инфраструктура удалённых вызовов процедур. Сервис и сообщения обычно описываются в .proto-файле, а инструменты генерируют клиентский и серверный код. По умолчанию используется Protocol Buffers, транспорт основан на HTTP/2. Поддерживаются одиночный запрос и ответ, поток ответов сервера, поток запросов клиента и двунаправленный поток.

REST — архитектурный стиль, в котором взаимодействие строится вокруг ресурсов, их представлений и единообразного интерфейса. В HTTP API обычно применяются GET, POST, PUT и DELETE; JSON распространён, но не обязателен. gRPC организует взаимодействие как вызов именованного метода, а REST — как операцию над ресурсом. gRPC удобен для строго типизированных внутренних сервисов и потоковой передачи. REST API проще проверять обычными HTTP-инструментами и использовать в браузере. REST не ограничен HTTP/1.1 и не обязан передавать только JSON; производительность зависит от конкретной реализации.

## 10 Репозиторий

Ссылка на GitHub: _____

## 11 Вывод

Разработан REST-сервис с файловой базой H2 и двумя таблицами. CRUD-операции разделены по слоям приложения, входные данные проверяются, а ответы сообщают об успехе или ошибке операции. Автоматические и ручные проверки подтвердили корректность основных сценариев, обработку ошибок и сохранение данных после перезапуска.

## 12 Источники

H2 Database Features — https://h2database.com/html/features.html

API-first — https://www.postman.com/api-first/

Enterprise Integration Patterns — https://www.enterpriseintegrationpatterns.com/patterns/messaging/Chapter1.html

Introduction to gRPC — https://grpc.io/docs/what-is-grpc/introduction/